

从“能用”到“会用” | 如何写好一个 Skill

本文作者：Jiaqi, TRAE 技术文档工程师

 之前带大家了解了 Skill 的原理和应用，本文继续带领大家从“能用”到“会用”，如何写好一个 Skill 至关重要。

本文为 Skill 设计与开发的一站式最佳实践指南，可助力你：

- 系统厘清 Skill 的定义，规避常见认知误区；
- 熟练掌握高命中率、高稳定性 Skill 的设计标准与核心原则；
- 掌握“评测驱动、失败优先”的工程化 Skill 构建与迭代全流程；
- 掌握可维护、可扩展的 Skill 设计技巧；
- 依托 AI 高效完成 Skill 的创建、迭代与优化；
- 凭借反模式检查清单，精准规避 Skill 开发中的典型问题。

什么是 Skill？

定义

一个 Skill 是一份清晰、严谨、可执行的指令文档，用于明确告诉模型——在什么条件下（When），按照哪些步骤（How），产出什么结果（What）。

常见认知误区

在编写 Skill 之前，建议先了解以下几个常见误区。这些问题往往会直接导致 Skill 难以被正确触发，或在执行过程中表现不稳定。

• 误区一：Skill 等同于一段 Prompt

Skill 并不是一次性的对话提示。它是一个可长期复用、输入输出明确的能力模块，强调的是稳定、确定且易于工程化维护。而 Prompt 更偏向临时性、探索性和即兴交互，两者在设计目标和工程要求上完全不同。

• 误区二：Skill 是写给人看的文档

Skill 的目标不是“解释原理”，而是“下达指令”。`SKILL.md` 文件的内容应使用模型可解析的结构化语言，明确约束其行为边界，并精确描述何时使用（When）、如何执行（How）、输出结果（What）。

• 误区三：Skill 越复杂越强大

复杂度并不与 Skill 的能力强度挂钩。模型的推理与决策成本是显著的。职责单一、边界清晰的 Skill，更容易在正确的时机被选中并稳定执行。过于复杂的 Skill 反而会降低命中率。



提示

模型的上下文窗口是有限且宝贵的公共资源。每一个被加载的 Skill 都在竞争有限的上下文资源。因此，Skill 文档应以最小必要信息为目标，避免冗余解释与不必要的背景铺垫。

Skill 的设计标准与原则

以下标准是构建高命中率、高稳定性 Skill 的基础。可将其作为设计与评审 Skill 时的检查清单。

精准的元数据内容

Skill 的元数据（name 和 description）是模型发现和识别 Skill 的入口，其设计直接影响触发准确率。

元数据字段	规范	示例
name	用于识别 Skill。应遵循以下规范： • 简洁、唯一的标识符 • 使用小写字母、数字和连字符（-） • 推荐使用动名词（Gerund form） • 长度不超过 64 个字符	• 好的示例： ◦ <code>running-tests</code> ◦ <code>deploy-microservice</code> ◦ <code>database-migration</code> • 坏的示例： ◦ <code>test-helper</code> （语义模糊） ◦ <code>data-skill-v2</code> （冗余且含版本信息） ◦ <code>deployService</code> （命名不规范）
description	用于描述 Skill 的能力及适用场景。应遵循以下规范： • 使用第三人称，从模型视角描述 • 包含核心功能与触发时机的关键词 • 长度不超过 1024 个字符	• 好的示例 “Review code for quality, correctness, and maintainability. Use when evaluating pull requests, refactoring existing code, or when the user asks for feedback on implementation details, edge cases, or potential bugs” . • 坏的示例：

		“I can help you review code”（第一人称） “Helps with code review”（缺乏触发时机）
--	--	---

指导方式的自由度分级

根据任务复杂度与容错要求，合理控制对模型的约束强度。

自由度等级	适用场景	指导方式	示例
高	- 存在多种有效方法 - 模型的决策依赖上下文	提供启发式策略（给原则）	代码审查：先看安全性，再看可读性 代码块 <pre>1 --- 2 name: code-review 3 description: 当用户需要对代码进行审核时，基于代码实现与通用开发规范，分析逻辑正确性、可维护性和潜在风险，并给出改进建议。</pre>
中	- 存在首选模式 - 允许一定程度的变通 - 行为受配置参数影响	提供模板/伪代码（给框架）	报告生成：按"摘要-分析-建议"结构 代码块 <pre>1 --- 2 name: report-generator 3 description: 当用户需要生成报告时，按照“摘要-分析-建议”的结构整理信息，输出清晰、条理化的报告内容 4 template: 5 摘要： 6 - 简要概述核心信息或问题点 7 8 分析： 9 - 详细分析背景、原因、数据或逻辑 10 - 列出关键发现和关联因素 11 12 建议： 13 - 针对分析结果提出具体可行的改进方案或行动建议 14 - 如有优先级或风险提示，可附上</pre>

低	- 操作脆弱且易错。 - 一致性至关重要。 - 必须遵循特定序列。	提供可执行的脚本 (给代码)	数据库迁移：按固定顺序执行脚本 代码块 1 --- 2 name: database-migration 3 description: 当用户需要执行数据库迁移时，按预定义顺序执行 SQL 或迁移脚本，确保数据和结构一致性 4 template: 5 迁移计划： 6 1. 准备阶段： 7 - 备份现有数据库 8 - 验证目标环境配置 9 2. 执行阶段（按顺序执行脚本）： 10 - 脚本 001_create_users_table.sql: 创建用户表 11 - 脚本 002_add_email_index.sql: 为用户表添加邮箱索引 12 - 脚本 003_update_roles.sql: 更新角色字段数据 13 3. 验证阶段： 14 - 检查数据完整性 15 - 验证关键功能是否正常 16 4. 回滚（可选）： 17 - 如出现异常，按回滚方案恢复数据库
---	---	-------------------	---

五个核心标准

标准	描述	示例
边界明确	模型最容易犯的错误不是“不知道怎么做”，而是“不知道什么时候该做”。Skill 的触发必须有明确的正向条件和负向条件，否则命中率会很低。	✖ 边界不清晰： Use this skill when: - 用户想让流水线跑一下测试。 - PR 有代码改动或文档改动。 Do NOT use this skill when: - 用户只是在看 PR。

		✓ 边界清晰： Use this skill when: - 用户意图是触发 CI/CD 流水线执行单元测试。 - PR 状态为“待合并”，需要执行自动检查或 lint 校验。 Do NOT use this skill when: - 用户只是查看测试报告或 CI/CD 状态。 - PR 内容没有代码改动，仅修改文档或注释。
输入输出结构化	与模型沟通时，需要定义双方都能理解的“共同语言”，避免模糊描述。推荐用类似函数签名的方式明确 Input 和 Output，保证可解析性。	✗ 模糊描述： “帮用户跑测试并返回结果。” ✓ 结构化定义： 代码块 1 Input: 2 - prId: string # PR 编号 3 - branch: string # 分支名称 4 - runTests: boolean # 是否执行单元测试 5 Output: 6 - success: boolean # 是否成功执行 7 - testReport?: object[] # 测试报告 (可选) 8 - errorMessage?: string # 错误信息 (失败时返回)
步骤明确可执行	Skill 的核心是“步骤”，必须是指令式、具体动作，而不是概括性描述，确保模型可以按步骤执行。	✗ 描述性语言： “检查 PR，运行测试，然后更新状态。” ✓ 指令式步骤： 代码块 1 Steps: 2 1. Validate PR: 检查 prId 和 branch 是否有效。 3 2. Checkout branch: 切换到指定分支。

		4 3. Run tests: 根据 runTests 参数执行单元测试。 5 4. Collect results: 收集测试结果。 6 5. Update PR status: 将测试状态回写到 PR。 7 6. Notify user: 若测试失败, 返回错误信息。
失败策略完 备	模型在失败时容易自由发挥, 可能产生不可预期的行为。必须明确定义“失败路径”, 告诉模型在不同失败情况下如何处理。	失败策略示例: 代码块 1 On Failure: 2 - Validation fails: 返回 400 Bad Request, 并附上具体错误信息。 3 - Test execution fails: 自动重试一次, 仍失败则返回“单元测试未通过, 请检查日志”。 4 - CI/CD 服务不可用: 重试最多 3 次, 仍失败则记录日志并通知管理员。

构建与迭代 Skill 的最佳流程

Skill 的开发是一个以失败为起点、评测为牵引, 持续迭代优化的工程化过程。

评测并非事后的验证环节, 而是 Skill 设计的前提; Skill 也非基于假设的规则集合, 而是针对已暴露问题的最小化解决方案。

遵循“评测驱动、失败优先”的原则，能确保 Skill 在实际使用场景中拥有清晰的能力边界、稳定的运行表现，以及可回归的质量保障体系。

第一步：建立无 Skill 基线，识别真实问题

在编写任何 Skill 之前，应先不使用 Skill，直接让模型执行目标任务，作为基线对照。

重点观察并记录以下问题：

- 模型在哪些情况下表现不稳定或结果不可复现；
- 哪些输入会引发歧义、误解或走偏；
- 模型是否在错误的时机尝试“主动帮忙”。

这些失败点和不确定行为，本质上就是 Skill 需要解决的真实能力缺口，也是后续评测用例的来源。

第二步：以“失败优先”为原则，定义评测用例

明确核心问题后，需优先编写评测用例，而非直接开发 Skill。评测是约束，Skill 是落地实现。脱离评测约束的 Skill，本质上是在放大模型的行为不确定性。

评测的核心作用是约束 Skill 的行为边界，明确以下信息：

- 什么场景下属于 Skill 的正确使用范围？
- 什么场景下 Skill 必须拒绝执行或判定为失败？
- 什么样的输出结果才算稳定可用？

推荐做法：

- 针对已识别的问题，设计 3-5 个具体、可复现的评测用例；
- 每个用例均需明确“通过 / 失败”的判定标准；
- 优先覆盖模型最易误用 Skill 的场景。

第三步：编写最小化 Skill，明确最短成功路径

在评测已经存在的前提下，开始编写 Skill。

此阶段不追求覆盖所有情况，而是只编写刚好能够通过当前评测的最小规则集合，重点关注三个要素：

- **明确失败条件（When NOT to use）**

将评测中识别的失败场景，显式写入 Skill 中，作为第一层防护，避免 Skill 被误触发或滥用。

- **定义最短成功路径**

清晰描述 Skill 最简单、最核心的执行流程，确保最精简的有效输入能得到可预测的稳定输出。

- **保持职责单一**

单个 Skill 仅解决一个明确问题，对应一个核心动作，避免在早期引入额外复杂度。

这一阶段的 Skill，是评测结果的直接产物，而非凭经验预判的方案。

第四步：补充边界条件与结构化示例

当最短成功路径能稳定通过评测后，再逐步扩展 Skill 的适用范围。

完成以下三项核心工作：

- 补充更多边界场景及对应的行为约束；
- 明确 Skill 输入（Input）、输出（Output）的结构化定义；
- 补充关键的输入、输出示例，帮助模型对齐行为预期。

核心原则：所有新增规则，均需对应新增或已有评测用例，避免在无评测支撑的情况下将 Skill 复杂化。

第五步：评测回归与持续迭代

Skill 的迭代，需始终与评测结果强绑定，遵循以下规则：

- 新增评测用例 → 推动 Skill 的增量修改；
- 任意修改 Skill → 必须通过已有评测的回归验证；
- 若评测未通过，优先简化 Skill，而非盲目叠加新规则。

通过持续对比模型在“无 Skill 基线”与“当前 Skill + 评测”中的表现，可验证该 Skill 是否真正提升了模型执行目标任务的成功率与稳定性。

第六步：结合真实使用路径进行校准

评测仅能覆盖已知问题，在真实场景中使用 Skill 时，可能会暴露更多新的问题。

在 Skill 实际使用过程中，需持续观察以下情况：

- 模型是否在非预期场景下误触发 Skill？
- 模型执行 Skill 时，是否遗漏关键参考文件或上下文？
- 模型是否会反复读取某一段内容，形成隐性依赖？

上述这些信号，均需作为新的评测输入，重新进入 Step 2，形成 Skill 迭代的闭环。

让 Skill 可维护与可拓展

渐进式披露

`SKILL.md` 应当作为 Skill 的入口和导航，而不是一个包罗万象的大文件。详细的参考资料、示例、脚本或文档应拆分成独立文件，从而减轻模型初次加载的负担，让信息按需流动。

- **信息架构原则：从简单到复杂**

一个 Skill 的目录可以随着功能扩展逐步演化：从单一文件 → 多个参考文件和脚本组成的结构。通过渐进式披露，模型能快速抓住核心信息，再深入了解细节。

• 最佳实践

- **保持 SKILL.md 简洁**：主体内容尽量控制在 500 行以内，只包含必要信息。
- **避免深度嵌套**：所有引用文件最好直接由 `SKILL.md` 链接，保持一层引用深度，避免链式引用（A → B → C），防止模型只读取部分内容。
- **为长文件添加目录**：对于超过 100 行的参考文件，在文件顶部添加一个目录（Table of Contents），帮助模型快速了解文件结构。

• 示例

以下为一个渐进式披露的 SKILL.md 示例：

代码块

```
1  # SKILL.md
2
3  ## 基础用法
4  描述如何触发 CI/CD 流水线：
5  - 检查 PR 状态
6  - 执行单元测试
7  - 更新 PR 测试状态
8
9  ## 高级功能
10  详细说明请参见 `ci-advanced-features.md`：
11  - 并行执行多分支测试
12  - 条件触发不同类型的测试
13  - 自定义失败处理策略
14
15  ## API 参考
16  所有方法与参数说明请参见 `ci-api-reference.md`：
17  - startPipeline(prId: string, branch: string)
18  - getPipelineStatus(pipelineId: string)
19  - cancelPipeline(pipelineId: string)
```

工作流与反馈闭环

对于包含多个步骤、且中间结果会影响最终质量的复杂任务，仅提供最终目标是不够的。必须显式定义工作流（Workflow）和 检查清单（Checklist），引导模型按步骤执行，并在关键节点建立“验证 → 修正 → 再验证”的反馈闭环。

工作流负责约束任务执行顺序，检查清单负责追踪任务的执行状态和质量。两者结合可以显著降低遗漏和跑偏的风险。

• 分析类任务的工作流

即使不涉及代码，分析类任务同样适合使用工作流。检查清单可以帮助模型明确“当前做到哪一步”、“是否可以进入下一步”。例如：

代码块

```
1  ## 技术方案评估工作流
2
3  在执行前复制以下清单，并在每一步完成后显式标记状态。
4
5  - Step 1: 明确业务目标与技术约束（性能、成本、时限）
6  - Step 2: 列出所有可行的技术方案
7  - Step 3: 从复杂度、可维护性、风险角度逐一评估
8  - Step 4: 对关键差异点进行对比分析；（反馈闭环）若发现关键信息不足，应返回 Step 2 或 Step 3 补充分析
9  - Step 5: 给出结论性建议，并说明取舍理由；（反馈闭环）若结论无法支撑目标约束，应重新审视 Step 1 的前提条件
```

• 代码类任务的工作流

代码类任务往往伴随不可逆或影响范围较大的操作，例如重构、依赖升级或配置变更。通过“Plan → Validate → Execute”模式，可以有效降低误操作风险。例如：

代码块

```
1  ## 依赖版本升级工作流
2
3  - Step 1 (Plan) :
4    - 识别需要升级的依赖及当前版本
5    - 阅读目标版本的 Release Notes 与 Breaking Changes
6  - Step 2 (Plan) :
7    - 更新依赖配置文件（如 package.json / go.mod）
8    - 标注可能受影响的模块
9  - Step 3 (Validate) :
10   - 执行依赖冲突检查与静态构建（运行 dependency_check.sh）
11   - 确认无版本冲突或构建失败
12   - （反馈闭环）若校验失败，必须回退到 Step 2 调整依赖配置
13 - Step 4 (Execute) :
14   - 安装新版本依赖
15   - 运行完整测试集
16 - Step 5 (Validate) :
17   - 检查核心功能是否受影响
18   - 对比升级前后的构建与运行结果
19   - （反馈闭环）若出现回归问题，应回滚升级并记录风险点
```

可执行脚本的加固原则

当 Skill 依赖可执行脚本时，脚本的健壮性应始终优先于代码的巧妙性。

Skill 本身不会理解或阅读你的代码逻辑，它只感知输入与输出。一旦脚本行为不可预测，模型就只能猜测，最终导致不稳定或错误的调用结果。因此，脚本必须做到：失败可预期、输出可理解、参数可解释。

• **显式处理错误，而不是让模型猜**

不要将异常直接抛给模型处理。脚本应覆盖常见错误场景，并将技术异常转化为可理解、可决策的输出。

实践要点：

- 捕获常见异常（如文件缺失、权限不足、配置错误）
- 为每类错误返回清晰的错误原因和下一步建议

示例：配置文件校验脚本

代码块

```
1  ERROR: Config file not found: ./deploy.yaml
2  HINT: Please check whether the file path is correct or run init-config.sh
    to generate a default config.
```

• **输出自解释的日志与验证结果**

脚本的输出本身就是模型的“上下文”。一个好的脚本不仅说明发生了什么，还说明为什么会这样，以及接下来可以怎么做。

实践要点：

- 成功路径和失败路径都要有明确输出
- 验证类脚本应明确列出通过项与失败项

示例：构建环境检查脚本

代码块

```
1  CHECK FAILED: Node.js version mismatch
2  - Required: >= 18.0.0
3  - Detected: 16.14.0
4
5  VALID OPTIONS:
6  1. Upgrade Node.js to a supported version
7  2. Switch to a compatible build image
```

• **避免魔法数字，让参数“有来由”**

脚本中的常量（如 `TIMEOUT = 30`）如果缺乏解释，模型和人都无法判断它是否合理。任何影响行为的数值，都应该是可解释、可调整的。

实践要点：

- 为常量添加语义化名称
- 说明数值来源或设计依据
- 必要时允许通过参数覆盖默认值

示例：部署等待脚本

代码块

```
1 TIMEOUT_SECONDS = 30 # Wait up to 30s because service startup usually
  completes within 10-20s
```

或在输出中体现：

代码块

```
1 INFO: Waiting for service to become healthy (timeout: 30s)
```

巧妙使用 AI 来创建和迭代 Skill

在创建和迭代 Skill 的过程中，可以多用 AI。你负责定义问题和验收结果，AI 负责反复试错、总结规律并封装成可复用的 Skill。

阶段一：初次创建（从具体任务中抽象）

让 AI 从真实任务中抽象出创建 Skill 所需的信息，然后创建初版 Skill。

1. 让 AI 直接执行真实任务

提供完整目标与上下文，让 AI 自行尝试完成任务。执行过程中的追问、走偏和修正，本质上就是一次“隐式评测”。

2. 引导 AI 进行结构化复盘

任务完成后，要求 AI 从以下维度复盘：

- 成功执行任务的完整步骤；
- 任务执行过程中的不确定性与失败点；
- 可抽象的固定流程与判断逻辑；
- 该流程和判断逻辑的适用场景与不适用场景。

3. 基于复盘生成 Skill 初稿

要求 AI 按 Skill 规范生成 SKILL.md，明确：触发条件（When）、如何执行（How）、输出结果（What）、预设失败策略。

4. 人工快速评审并入库

你只需关注边界是否合理、步骤是否可执行，其余交由 AI 完成。确认后，让 AI 调用 `skills-creator` 正式创建该 Skill 并添加至你的项目。

阶段二：持续迭代（从使用反馈中优化）

当 Skill 在使用中暴露新问题时，对其进行优化。

1. 对齐偏差来源

引导 AI 分析问题源自 When / What / How 中的哪一部分。

2. 直接修改 Skill 并验证回归

更新 `SKILL.md`，并同时验证：

- 确保本次迭代未破坏原有的“黄金路径”。
- 确保新发现的错误场景已被覆盖。

在 TRAE 中创建 Skill 的其他方式

除了通过 AI 对话创建 Skill 外，你还以手动创建或导入外部 Skill。详细说明点击[阅读原文](#)参考官网文档。

附录：反模式检查清单

本表列出了在 Skill 开发中常见的反模式（Anti-Patterns），以及相应的原因、正确示例和错误示例。通过遵循这些规范，可以提升 Skill 的稳定性、可维护性和跨平台兼容性，避免模型误用或运行失败。该 Checklist 可作为开发、评审和迭代 Skill 时的快速参考指南。

反模式	原因	正确示例	错误示例
使用 Windows 风格路径	跨平台兼容性差，Unix/Linux 系统会报错。	<code>configs/deploy.yaml</code>	<code>configs\deploy.yaml</code>
提供过多选择	过多选项会令模型困惑，增加决策成本与不确定性。 当存在多个选项时，Skill 必须指明一个“默认路径”。	"默认使用 PostgreSQL 作为数据库（适用于常规生产和测试环境）。仅在对兼容性有特殊要求或客户已有 MySQL 环境时，才使用 MySQL"。	"你可以用 PostgreSQL，或者 MySQL，或者 SQLite，或者 Oracle..."。
包含时效性信息	信息容易过期，导致 Skill 不可用。	将旧版本配置放入 <code>deprecated/</code> 文件	"如果在 2025 年 8 月之前，请使用旧 API；之后

		夹，并注明“不再推荐使用”。	请使用新 API。”
术语不一致	增加模型的理解成本，降低 Skill 可用性。	始终使用 "Service Endpoint"。	混用 "Service Endpoint"， "API URL"， "Endpoint Path"。

参考资料

<https://platform.claude.com/docs/zh-CN/agents-and-tools/agent-skills/best-practices>

阅读原文：<https://docs.trae.cn/ide/skills#8be9a856>

更多信息请关注TRAE企业版服务号👉



微信搜一搜



TRAE企业版